Minitest is part of the Ruby Standard Library and is available everywhere you might use Ruby (1.9 and up). It has a straightforward syntax that is the Ruby translation of the original SUnit and JUnit libraries (for Smalltalk and Java, respectively), and it is the default alternative for a new Rails project. Minitest is used by the Rails core team to test Rails and has Rails-specific extensions.

RSpec is a separate testing tool designed to support an emphasis on specifying behavior rather than implementation, sometimes called behavior-driven development (BDD). Rather than using terms like "test" and "assert," RSpec uses "expect" and "describe." BDD emphasizes setting expectations about the code not yet written rather than assertions about code in the past.

RSpec has a quirky syntax that sometimes depends on metaprogramming and that people often have strong feelings about, both positive and negative. It's more flexible, which means it's more expressive but also more complicated, and it has a larger ecosystem of related tools. I believe that more Rails applications use RSpec than Minitest, though this is also disputed.

The primary testing tool used in this book is RSpec, because after going back and forth quite a bit, I decided that its expressiveness makes it easier to work with over the course of the entire book, even if it has a slightly steeper learning curve. It's also the tool that I use in my own day-to-day work, which counts for something. That said, there's a whole chapter on Minitest, and I'll discuss most of the extra tools in a way that references both RSpec and Minitest. Most RSpec examples in the downloadable code have a corresponding Minitest version.

That leads to a more general point: sometimes the best practice for learning isn't the best practice for experts. In some cases in this book, I'll use relatively verbose or explicit versions of tools or tests to make it clear what the testing is trying to do and how.

Changes in This Edition

A lot has changed in the Rails testing world over the past seven years, even if the general principles have stayed more or less the same. The entire community, including me, has had more years of experience with these tools, building bigger and better applications, learning what tools work, what tools scale, and what tools don't.

Rails 5 Test Prescriptions has several updates from the Rails 4 version: